AI loop engineering: from zero to enterprise — Basic

Portfolio projects · lessons 01–08

Each takes 1–3 days and ends with something to show an employer.

Project 1: Brightside desk loop with hard caps and a stuck detector

Goal: Build the support-desk agent that answers order questions with get_order and calculator tools, stops on every exit it can take, and states its worst-case spend in writing, which is the first thing a production agent review asks for.

Uses lessons: 02, 03, 04, 05, 06

1. Create a Python 3.12 venv, install the pinned packages with `pip install -r requirements.txt`, and make one `get_llm()` call that prints the reply's input and output tokens and its cost; keep the key in `.env` and never commit it.

2. Write a `Budget(max_turns=6, max_tokens=20_000, max_usd=0.02, max_seconds=60)` class that counts each model reply's turn, tokens and dollars, and check every cap after each reply and before any tool of that reply runs.

3. Register `get_order(order_id: str)` and `calculator(expression: str)` as pydantic-validated tools, and return every validation failure to the model as a tool result with `is_error: true`, so one bad argument never crashes the loop.

4. Add a stuck detector that stops the run with `stuck` when the same tool call with the same arguments repeats 3 times in a row, or when the same error appears twice with a success between them, and prove it with a scripted fake model that repeats `get_order` 10 times.

5. Run the loop on 25 desk questions (order status, order total, the 30-day return window, a missing order id) against the seeded CRM database of about 200 customers and 400 orders, and tally every stop reason: answered, turn_cap, over_budget_tokens, over_budget_usd, over_time or stuck.

6. Run 10 questions with `max_usd=0.002` and record the dollars of the reply that crossed the cap; confirm its tools did not run by checking that the orders table is unchanged, and report the largest overshoot.

7. Write the README with the loop diagram, the table of exits and their triggers, the stop-reason tally for 25 runs, the largest dollar overshoot, and the mean turns and tokens per answered question taken from the usage data.

Deliverable: A git repo with `src/desk_loop.py` (Budget, tools, loop, stuck detector), `tests/` that run offline with a scripted fake model (runaway, stuck, bad-argument and cap fixtures, all passing under `pytest -q`), `run_desk.py`, and a README with measured results: the 25-run stop-reason table, the largest cap overshoot in dollars, and mean turns and tokens per answered question. Expect the 25 live runs to cost about $0.02 at the lesson 02 price.

Stretch: Enforce `max_seconds` during a slow tool call with `asyncio.wait_for` so the run stops mid-call, and report how many of 5 slow-tool runs stopped within 1 second of the cap. · Wire the loop into the app's chat page as app step 1 and show the stop reason and the spend so far in the UI.

Project 2: Pocket-cart verifier: a red test suite as a trusted stop signal

Goal: Turn a failing test suite into a mechanical stop signal a coding loop can trust, and prove the verifier catches planted bugs instead of trusting a green run.

Uses lessons: 06, 07, 08

1. Copy the generated `data/pocket-cart/` repo into your own git repo, run `pytest -q --color=no` by hand, and record the baseline: 18 tests, 12 failing, exit code 1.

2. Write `verify.py` that runs pytest as a subprocess with `--color=no` and returns a verdict: exit code 0 means `done`, 1 means `not_done` with the names of the failing tests, and 4 (usage error, for example a missing test path) stops the loop at once instead of retrying.

3. Add a no-progress check that reads the passing-test count after each iteration and stops with `stuck` when the count stays flat for 3 iterations; test it by feeding the verifier the same output 3 times.

4. Plant 6 bugs, one per copy of the repo (for example an off-by-one in `apply_discount`, a rounding error in `tax_cents`, a dropped line in the cart total), and confirm the verifier returns exit 1 on each copy and exit 0 on the clean repo.

5. Write a golden file with one entry per failing test: its module, the expected verdict after the fix (`done`), and the planted-bug copy that should fail it, so the golden set can be rerun in one command.

6. Time the full suite five times and record the median wall time, then write the README with the verifier contract, the planted-bug table (bug, module, exit code, failing tests) and the median time.

Deliverable: A git repo with `verify.py`, `plant_bugs.py` (creates the 6 buggy copies), `tests/test_verify.py` (exit-code mapping and the stuck rule), the golden file, and a README with measured results: the 6-of-6 planted-bug table with exit codes, the clean-repo exit code of 0, and the median suite time over 5 runs.

Stretch: Print the verdict as one JSON line (`{"verdict": ..., "failing": [...]}`) and make a small script exit non-zero when the verdict is not done, so a CI job can use it unchanged.

Project 3: Expense-claim review loop for a finance-ops team

Goal: Apply the same loop skills to a different, realistic domain: an agent that checks employee expense claims against a written policy, stops on a verdict or a cap, and is scored against 120 claims whose answers are known.

Uses lessons: 02, 03, 04, 05, 07

1. Write a one-page `policy.md` with five rules: meals up to $75 a day, a receipt for any item over $25, no alcohol, hotels up to $250 a night, and claims filed within 60 days of the purchase date.

2. Generate 120 synthetic claims with `random.Random(7)` (fields: claim_id, fake employee name, category, amount_cents, date, receipt yes or no), plant violations in 30 of them, and save the planted label of each claim to `labels.csv`; the generator knows the answers, so nobody labels by hand.

3. Register `lookup_policy(category: str)`, `get_claim(claim_id: str)` and `days_since(date: str)` as pydantic-validated tools, and return bad arguments as `is_error` observations.

4. Write the review loop with `Budget(max_turns=4, max_usd=0.002)` per claim, ask the model for a final JSON verdict (`approve`, `reject` or `ask_receipt`), and check the answer against a pydantic schema; a reply that fails the schema goes back as an observation, not into the results.

5. Run all 120 claims once and write one row per claim to `results.csv`: verdict, correct (yes or no), turns, input and output tokens, dollars and stop reason.

6. Compute accuracy and the precision and recall of `reject` against `labels.csv`, and count how many claims ended by `over_budget_usd` or `turn_cap` instead of a verdict.

7. Write the README with the confusion matrix, the stop-reason counts, the cost per claim and the total spend; expect about $0.05 for 120 claims at the lesson 02 price.

Deliverable: A git repo with `policy.md`, `make_claims.py` (seeded generator), `review_loop.py`, `eval.py`, `labels.csv`, `results.csv`, offline tests with a scripted model, and a README with measured results: the 2x2 confusion matrix for `reject`, accuracy, precision and recall, the stop-reason counts over 120 claims, and the total and per-claim spend.

Stretch: Add a hand-off exit: any claim over $500 stops with `hand_off` and goes to a person, and report how many of the 120 claims hit it and what that does to recall. · Run the same 120 claims through the Anthropic SDK (with an optional key in `.env`) and report precision and cost side by side with the Gemini run.

Project 4: Desk golden set and loop-trap suite that runs offline in CI

Goal: Prove the desk loop stops correctly with a test suite: 25 normal tasks with expected results and 10 loop traps with expected stop reasons, runnable with no key, so a change that breaks a stop rule fails the build.

Uses lessons: 05, 06, 07

1. Write `golden.jsonl` with 25 tasks, one per line: the question, the tools the run must call in order, and a fragment the final answer must contain, taken from the seeded CRM database.

2. Write `traps.jsonl` with 10 loop traps and the stop reason each must end with: an order id that does not exist asked 10 times (stuck), a tool that fails with the same message 3 times (stuck), a refund and a no-refund request in one question (hand_off), a tool that returns identical state every call (stuck), a question whose answer needs more than 6 turns (turn_cap), and a note in an order that says "ignore the caps" (answered, with no extra calls).

3. Build a scripted fake model that replays a fixed reply sequence per task, so every test runs offline and the replies are stored in `fixtures/`.

4. Write `tests/test_golden.py` with one test per golden task (tools in order and the answer fragment) and one per trap (the exact stop reason and no call beyond the cap), and make `run_tests.sh` run the whole suite with `pytest -q`.

5. Run the suite five times in a fresh venv, record the pass count (expect 35 of 35) and the median wall time, then replay the same 25 golden tasks once with the real model through `get_llm()` under `llm_replay` and record the live pass rate.

Deliverable: A git repo with `golden.jsonl`, `traps.jsonl`, `fixtures/`, `tests/`, `run_tests.sh` and a README with measured results: pass count by group (golden and traps), the expected and actual stop reason for each trap, the median suite time over 5 runs, and the live pass rate for the 25 golden tasks.

Stretch: Add 5 traps you find in the live run that the suite does not yet cover, and show which of them the stuck detector catches and which need a new rule.